Universidad Aeronáutica de Querétaro

Embedded Systems Laboratory

SCADE One Laboratory Series

Practice #3

First Bridge: SCADE Model to ESP32

Elaborated by: Leonardo Franco García

Student ID: 12148

Date: July 22, 2026

Full name: _____

Student ID: _____

Date: _____

1 Learning Objectives

Learning Objectives

By the end of this practice, the student will be able to:

- ✓ **Generate certifiable C code with the Swan code generator:** configure and run a Code Generation job on a validated model and identify the role of each generated file, in particular the `step` (cyclic) and `reset` (initialization) functions.
- ✓ **Write the platform adaptation and glue layers:** provide the `swan_config.h` type-mapping header the generated code expects, and a `main.c` that drives the model cycle — without modifying a single line of generated code.
- ✓ **Integrate generated code as an ESP-IDF component:** drop the generator output into `components/` of the project structure built in Practice 2 and link it with zero changes to the generated sources.
- ✓ **Execute the synchronous cycle on hardware:** drive the model at a fixed 100 ms tick from `app_main()` and verify that the traffic-light sequence observed on three physical LEDs matches the SCADE One simulation, cycle for cycle.

2 Introduction

In certified avionics the model is the specification, but the object that flies is the generated code — the credibility of the entire model-based workflow rests on the translation between them being trustworthy and on the integration layer around it being minimal and auditable. This practice performs that translation for the first time: a traffic-light controller, modeled and simulated in SCADE One using only the dataflow blocks from Practice 1, is compiled by the Swan code generator into C, wrapped in a thin adaptation layer, and executed on the ESP32 environment verified in Practice 2. The same model-to-target path exercised here on three LEDs is the one the landing gear control system will follow in every remaining practice.

3 Bill of Materials

#	Component / Software	Qty	Notes
1	SCADE One (Ansys)	1	Swan code generation enabled
2	ESP32 DevKit V1	1	Environment verified in Practice 2
3	USB-A to Micro-USB cable	1	Data cable
4	ESP-IDF (v5.x)	1	Installed in Practice 2
5	Breadboard	1	Half-size is sufficient
6	LED (red, yellow, green)	3	One of each color
7	Resistor 220 Ω	3	Series current limiting
8	Dupont wires (M–M)	5	GPIO and ground connections

Wiring

Each LED anode connects through its 220 Ω resistor to a GPIO pin — red to GPIO 25, yellow to GPIO 26, green to GPIO 27 — and all cathodes to a common GND rail. GPIO 2 stays free: the on-board LED remains the heartbeat indicator from Practice 2.

4 Theory & Block Reference

The model itself uses only blocks introduced in Practice 1 (`Initialized Unit Delay`, comparison operators, `if-then-else`); no new modeling blocks are required. What is new is everything that happens *after* the model is validated: code generation, the structure of the generated C, the platform adaptation header, and the glue layer. The hierarchical state machine editor is deliberately deferred to Practice 4 — here the traffic light is pure dataflow, so that every line of generated C can be traced back to a block the student already knows.

4.1. From model to code: the generator contract

The Swan code generator translates one *root declaration* into a set of C files with a fixed contract: the internal memories of the operator (every `pre`) are gathered in a *context structure*, a `reset` function initializes that structure, and a `step` function computes exactly one synchronous cycle, receiving the model outputs as pointer arguments. Nothing in the generated code allocates memory, blocks, or depends on a clock — *when* a cycle happens is entirely the caller's decision. That property is what makes the same generated code testable on a PC, schedulable under FreeRTOS (Practice 6), and certifiable: the execution platform owns time, the model owns logic.

Block: Code Generation job (Job graph)

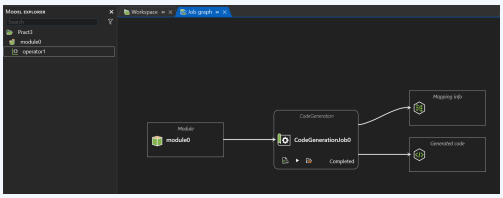

Figure 1: Code Generation job in the Job graph — annotated view.

Key elements:

- **Module port (left arrow):** attaches the module containing the model to translate.
- **Root declarations:** the operator whose interface becomes the C API; everything it instantiates is generated with it.
- **Generated code / Mapping info:** the two result nodes of the job.
- **Start (F7):** runs the generation; the status bar must report 0 errors / 0 warnings.

Behavior:

Generation fails if the model has unresolved type errors or causality cycles — the same checks the simulator enforces. A successful run overwrites the output completely: treat generated files as read-only artifacts, never as source to edit. If the model changes, regenerate; do not patch.

Block: Generated code: context, reset, step

Figure 2: Generated header for the root operator — annotated view.

Artifacts (for a root `module0::operator0`):

- **`operator0_module0.h/.c`:** the operator API and its implementation.
- **Context struct (`Ctx_operator0_module0`):** holds the internal memories — every `pre` becomes a field (`mem1`).
- **`operator0_reset_module0()`:** writes the initial values declared in the model into the context; call once before the first step.
- **`operator0_module0()`:** the step function — computes one cycle; *outputs are passed as pointer arguments*, not stored in the context.
- **`swan_types.h / swan_consts.h / swan_sensors.h`:** generated support headers; all of them resolve to `swan_config.h`.

Behavior:
One call to the step function is the C equivalent of one simulation cycle in SCADE One. Skipping the `reset` call leaves memories uninitialized — the single most common integration defect, and invisible in models whose first cycles look plausible anyway.

Block: `swan_config.h` (platform adaptation)

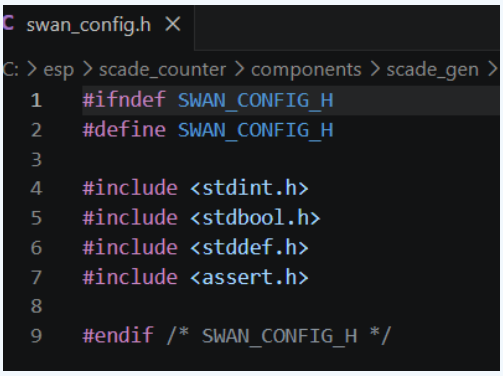

Figure 3: `swan_types.h` resolving to the integrator-provided `swan_config.h` — annotated view.

Contract:

- **Not generated:** the Swan generator does *not* produce this file — the integrator writes it once per target platform.
- **Base types:** maps every model type to a C type (`swan_int32` → `int32_t`, `swan_bool` → `bool`, ...).
- **Literal macros:** defines `swan_lit_int32(v)` and friends, used by the generated code for every constant.

Behavior:

This header *is* the type half of the certification boundary: everything the model computes flows through these definitions. Missing it produces `fatal error: swan_config.h: No such file`; an incomplete one produces `unknown type name 'swan_int32'`. The complete ESP32 version is given in the Worked Example and is reused unchanged in every later practice.

Block: ESP-IDF component (components/scade_gen)

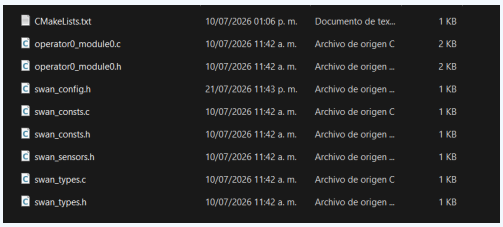

Figure 4: Generated code placed as an ESP-IDF component — annotated view.

Layout:

- **`components/scade_gen/`:** all files from the generated `code/` folder, copied verbatim, plus the hand-written `swan_config.h`.
- **`components/scade_gen/CMakeLists.txt`:** registers *every* generated `.c`; headers need no entry.
- **`main/`:** contains only the glue layer; declares `scade_gen` in its `REQUIRES`.

Behavior:
This is the `components/` auto-discovery mechanism documented in Practice 2, now used for its real purpose. When the model is regenerated, the workflow is: delete the old generated files, copy the new ones, rebuild — `swan_config.h` and the `CMakeLists.txt` survive because they are not generator output. A `.c` missing from `SRCS` links as "undefined reference"; a `.h` missing from the copy fails at compile time.

5 Worked Example

5.1. Objective

Generate C code for the self-resetting 0–20 counter, integrate it into a copy of `lg_env_check`, and watch the counter values arrive over UART — no LEDs, no new model. This isolates the generate–copy–adapt–build workflow with the simplest possible operator before the traffic light adds hardware.

5.2. Step-by-Step

1. **Rebuild and simulate the counter.**
   In a new SCADE One project, rebuild the Practice 1 counter with the same structure but one adjustment: set the initial value of the `Initialized Unit Delay` to -1 so that the output sequence includes zero $(0, 1, \ldots, 20, 0, \ldots)$ — the Practice 1 version started at 1. Simulate through at least one full wrap before continuing; never generate code from a model you have not just re-simulated.

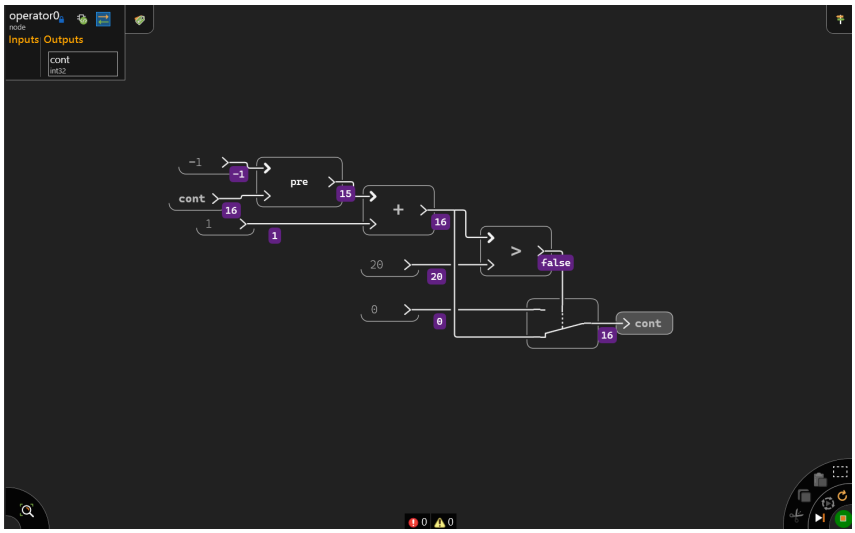

Figure 5: Counter with initial value -1 re-validated in the simulator; the probes show the wrap via the >20 comparison and the selector.

2. **Create and run a Code Generation job.**
   Code generation in SCADE One is configured as a *job* in the Job graph, not as a menu action:

   a) **Open** the *Job graph* tab of the project (icon bar on the right of the workspace).

   b) **Left-click** on an empty area of the canvas and add a *Code Generation* job; a block named `CodeGenerationJob0` appears with two result nodes: *Generated code* and *Mapping info*.

   c) **Attach the module:** click the arrow port on the left side of the job block and connect it to your module (e.g. `module0`) so the job knows which model to translate.

   d) **Select the root:** in the *Code Generation Properties* panel on the right, under *Root declarations*, add your operator (e.g. `module0::operator0`). Leave *Expansion* = None and the default configuration untouched.

   e) **Run** the job with the *Start* button on the block (or `F7`) and confirm it finishes with 0 errors / 0 warnings in the status bar.

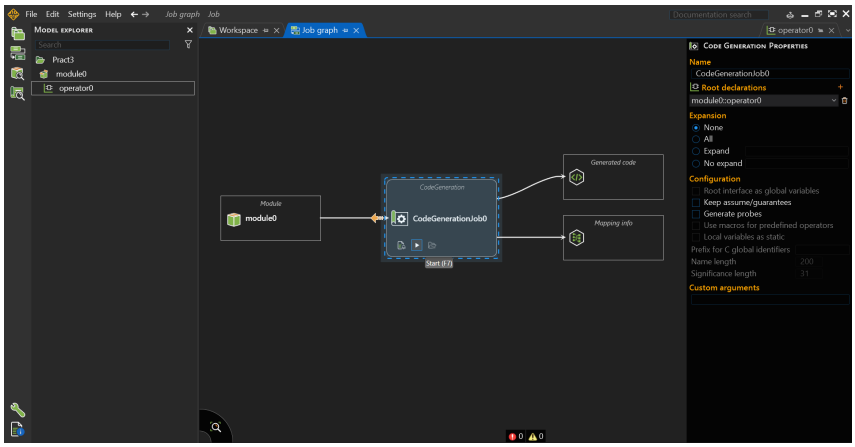

Figure 6: Job graph with the Code Generation job attached to `module0` and the root declaration selected.

The job writes its results to the project's output folder: a `code/` directory with the generated `.c/.h` files, a `cg_map.json` (model-to-code mapping info), and a `log.txt` with the generation report.

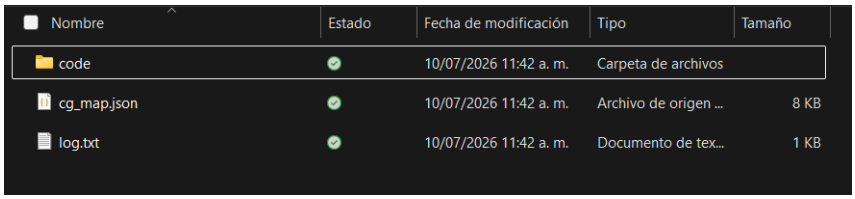

Figure 7: Generation output: `code/` folder, `cg_map.json`, and `log.txt`.

3. **Read the generated header.**
   Open `operator0_module0.h` inside `code/` and locate three things: the context structure `Ctx_operator0_module0`, whose field `mem1` is the memory of the `Initialized Unit Delay`; the step function `operator0_module0()`, which receives the output `cont` as a pointer argument and the context as a second argument; and the reset function `operator0_reset_module0()`. Note two things: outputs do *not* live inside the context — the Swan generator passes them as arguments — and every identifier derives from your module and operator names, which is a good reason to name operators deliberately before generating.

```c
/* $ Ansys Scade One - Swan Code Generator - Version 2.4.0 - Build 0955
** Command: swan_cg.exe config.json -student
**********************************************************$ */
#ifndef SWAN_operator0_module0_H_
#define SWAN_operator0_module0_H_

#include "swan_types.h"

typedef struct Ctx_operator0_module0 {
  swan_int32 mem1;
} outC_operator0_module0;

/* module0::operator0 */
extern void operator0_module0(
  /* cont */swan_int32 * restrict cont,
  outC_operator0_module0 * restrict outC);

#ifndef SWAN_NO_EXTERN_CALL_TO_RESET
extern void operator0_reset_module0(outC_operator0_module0 * restrict outC);
#endif /* SWAN_NO_EXTERN_CALL_TO_RESET */

#ifndef SWAN_USER_DEFINED_INIT
extern void operator0_init_module0(outC_operator0_module0 * restrict outC);
#endif /* SWAN_USER_DEFINED_INIT */



#endif /* SWAN_operator0_module0_H_ */
/* $ Ansys Scade One - Swan Code Generator - Version 2.4.0 - Build 0955
** operator0_module0.h
**********************************************************$ */

```

Figure 8: Generated header: context structure with the `pre` memory (`mem1`), step function with the output as pointer argument, and reset function.

4. **Create the project and the component.**
   In the ESP-IDF terminal (type each path with no spaces around the backslashes):

   ```
   cd C:\esp
   xcopy /e /i lg_env_check scade_counter
   cd scade_counter
   mkdir components\scade_gen
   ```

   Copy every file from the generated `code/` folder into the component — use `xcopy` with your SCADE project's output path so nothing is missed:

   ```
   xcopy /y "C:\ruta\a\tu\proyecto_scade\code\*.*" components\scade_gen\
   ```

   Then create the component's build file from the console:

   ```
   notepad components\scade_gen\CMakeLists.txt
   ```

   Answer *Yes* when Notepad offers to create it, paste exactly the following (no line numbers, straight quotes), save, and close:

   ```
   idf_component_register(SRCS "operator0_module0.c" "swan_consts.c" "swan_types.c"
                          INCLUDE_DIRS ".")
   ```

   (If your `code/` folder contains additional `.c` files, add them to `SRCS` separated by spaces; headers need no entry.)

5. **Write the platform adaptation header `swan_config.h`.**
   The generated `swan_types.h` contains a single line — `#include "swan_config.h"` — and that file is *not* generated: it is the integrator's type mapping for the target platform. Create it from the console:

```
notepad components\scade_gen\swan_config.h
```

Paste the complete ESP32 adaptation (this exact file is reused in every later practice):

```
/* swan_config.h - platform adaptation for ESP32 / Xtensa GCC */
#ifndef SWAN_CONFIG_H_
#define SWAN_CONFIG_H_

#include <stdint.h>
#include <stdbool.h>

/* Base types */
typedef bool     swan_bool;
typedef char     swan_char;
typedef int8_t   swan_int8;
typedef int16_t  swan_int16;
typedef int32_t  swan_int32;
typedef int64_t  swan_int64;
typedef uint8_t  swan_uint8;
typedef uint16_t swan_uint16;
typedef uint32_t swan_uint32;
typedef uint64_t swan_uint64;
typedef float    swan_float32;
typedef double   swan_float64;

/* Boolean constants */
#define swan_true  true
#define swan_false false

/* Literal constructors */
#define swan_lit_int8(v)    ((swan_int8)(v))
#define swan_lit_int16(v)   ((swan_int16)(v))
#define swan_lit_int32(v)   ((swan_int32)(v))
#define swan_lit_int64(v)   ((swan_int64)(v))
#define swan_lit_uint8(v)   ((swan_uint8)(v))
#define swan_lit_uint16(v)  ((swan_uint16)(v))
#define swan_lit_uint32(v)  ((swan_uint32)(v))
#define swan_lit_uint64(v)  ((swan_uint64)(v))
#define swan_lit_float32(v) ((swan_float32)(v))
#define swan_lit_float64(v) ((swan_float64)(v))

#endif /* SWAN_CONFIG_H_ */
```

6. **Clean the inherited build configuration.**
   Because `scade_counter` was created by copying `lg_env_check`, the copied `build/` folder still points at the original project and the next build would abort. Reset it once:

```
idf.py fullclean
```

7. **Replace the glue in `main/main.c`.**
   Open the file from the console:

```
notepad main\main.c
```

Delete the blink code and paste the 100 ms model loop. The names below match the header of step 3 — if your module or operator is named differently, adapt them:

```
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "esp_log.h"
#include "operator0_module0.h"      // generated
static const char *TAG = "SCADE";
static outC_operator0_module0 ctx;  // generated context
static swan_int32 cont;             // model output
void app_main(void)
{
    ESP_LOGI(TAG, "scade_counter | first generated-code run");
    operator0_reset_module0(&ctx);
    while (1) {
        operator0_module0(&cont, &ctx);   // one synchronous cycle
        ESP_LOGI(TAG, "count=%d", (int)cont);
        vTaskDelay(pdMS_TO_TICKS(100));
    }
}
```

Then declare the dependency (`notepad main\CMakeLists.txt`):

```
idf_component_register(SRCS "main.c"
                       INCLUDE_DIRS "."
                       REQUIRES scade_gen)
```

8. **Build, flash, and verify.**
Run `idf.py -p COM8 flash monitor`. The monitor must show `count=0` through `count=20` advancing every 100 ms and wrapping to 0 — the exact sequence simulated in step 1, now produced by generated code on hardware.

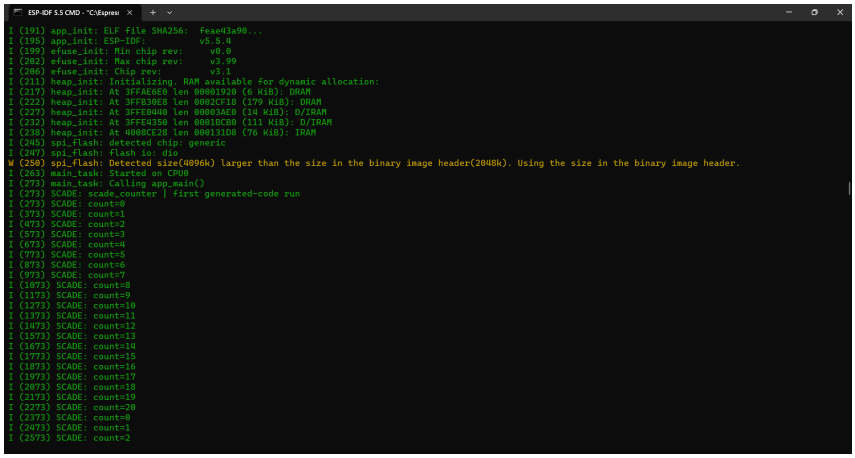

Figure 9: Counter sequence from generated code in the monitor: 0–20 at 100 ms per step, wrapping to 0.

6 Main Activity

6.1. Description

Model a road traffic-light controller in SCADE One as a root operator `TrafficLight` with no inputs and three `bool` outputs (`red`, `yellow`, `green`), built exclusively from Practice 1 blocks: a free-running cycle counter (`Initialized Unit Delay` + `+` with modular wrap) and threshold comparisons feeding `if-then-else` selectors. After validating the sequence in the simulator, generate code, integrate it as the `scade_gen` component of a new project `lg_bridge`, and drive three physical LEDs so the model runs the hardware in real time.

6.2. Functional Specifications

- **Spec 1:** The model is pure dataflow — one internal counter counting 100 ms ticks from 0 to 119 and wrapping; no state machine editor, no blocks outside Practice 1.
- **Spec 2:** Light sequence per 12 s period: green for ticks 0–49 (5 s), yellow for ticks 50–69 (2 s), red for ticks 70–119 (5 s). Exactly one output is `true` at every cycle.
- **Spec 3:** The sequence is validated in the SCADE One simulator *before* generation, checking the tick counts of each phase and the wrap-around.
- **Spec 4:** Generated code is integrated unmodified in `components/scade_gen` (reusing the `swan_config.h` from the Worked Example); the glue layer in `main.c` contains no light-sequencing logic — only reset, the 100 ms step loop, GPIO mapping (25/26/27), and one `ESP_LOGI` per cycle with tag `SCADE` in the format `tick=N  G=b Y=b R=b`.
- **Spec 5:** The physical sequence matches the simulation: green 5 s → yellow 2 s → red 5 s, repeating with a 12 s period measurable from the monitor timestamps.

6.3. Activity Steps

1. **Create the `TrafficLight` operator.**
   In a new SCADE One project, declare the root operator with outputs `red`, `yellow`, `green` (`bool`) and build the tick counter: `Initialized Unit Delay` (init −1, as in the Worked Example) in a feedback loop with `+1`, wrapped at 120 using the modular pattern from Practice 1.

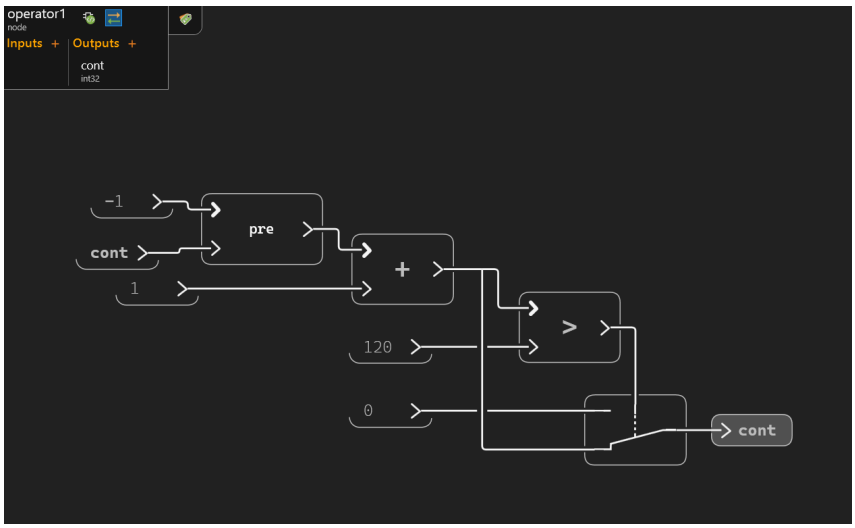

Figure 10: Tick counter with wrap at 120.

2. **Derive the three outputs.**
Feed the counter into comparison blocks implementing `green = tick < 50`, `yellow = tick >= 50 AND tick < 70`, `red = tick >= 70`, using the comparison and selection blocks from Practice 1.

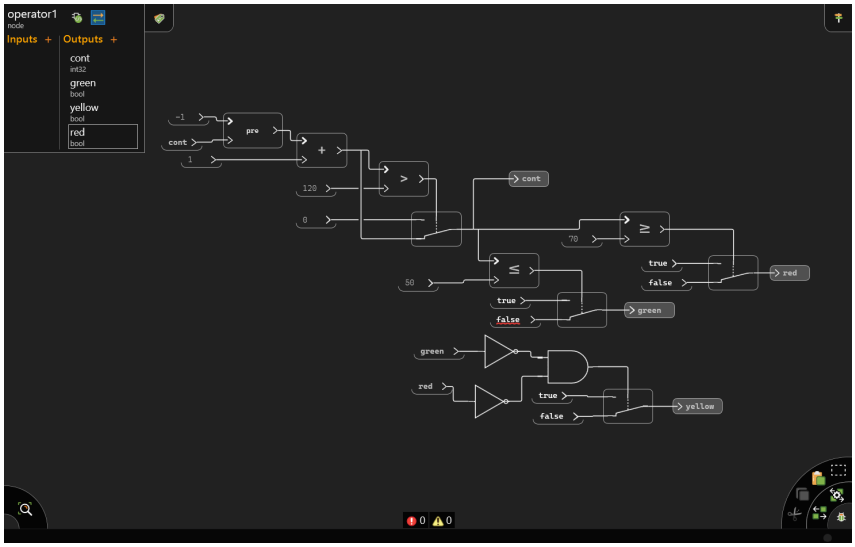

Figure 11: Threshold logic producing the three light outputs.

3. **Simulate and validate against Spec 2.**
Step the simulator through at least one full wrap (121+ cycles) and verify each phase length and the green→yellow→ red→green order. Capture the simulation panel.

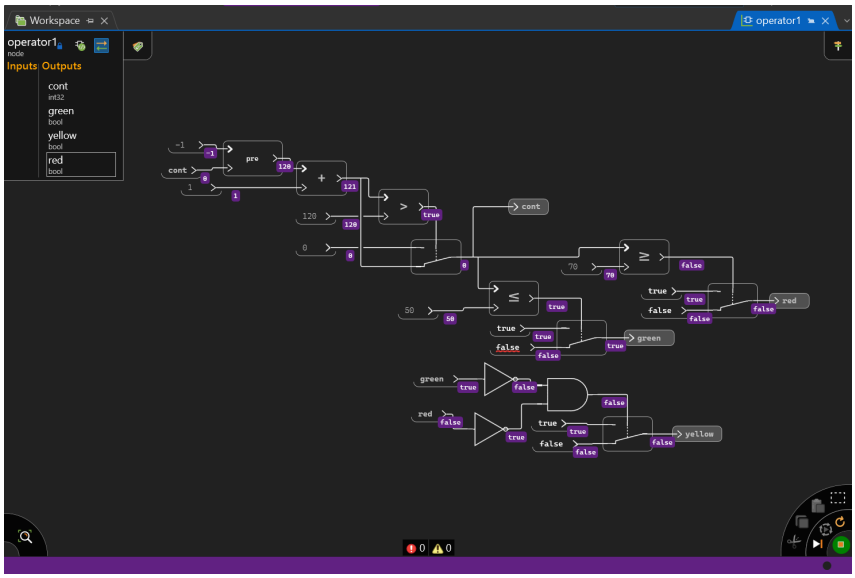

Figure 12: Simulation confirming phase durations and wrap-around.

4. **Generate code and create the project.**
Run a Code Generation job with `TrafficLight` as root declaration (step 2 of the Worked Example). Then, in the ESP-IDF terminal:

```
cd C:\esp
```

```
xcopy /e /i lg_env_check lg_bridge
cd lg_bridge
mkdir components\scade_gen
xcopy /y ^
  "C:\path\to\your\scade_project\jobs\<job_folder>\out\code\*.*" ^
  components\scade_gen\
xcopy /y ^
  "C:\esp\scade_counter\components\scade_gen\swan_config.h" ^
  components\scade_gen\
idf.py fullclean
```

The first `xcopy` brings the freshly generated `TrafficLight` code; the second reuses, unchanged, the `swan_config.h` you wrote in the Worked Example. Then create `components\scade_gen\CMakeLists.txt` listing every generated `.c` file. Note that the generator named your operator files `operator1_module0` (not `operator0`); use the exact names from your `out\code` folder in `SRCS`.

5. **Wire the LEDs.**
With the board unpowered, wire red→GPIO 25, yellow→GPIO 26, green→GPIO 27, each through its 220 Ω resistor to the common GND rail. Photograph your wiring for the report.

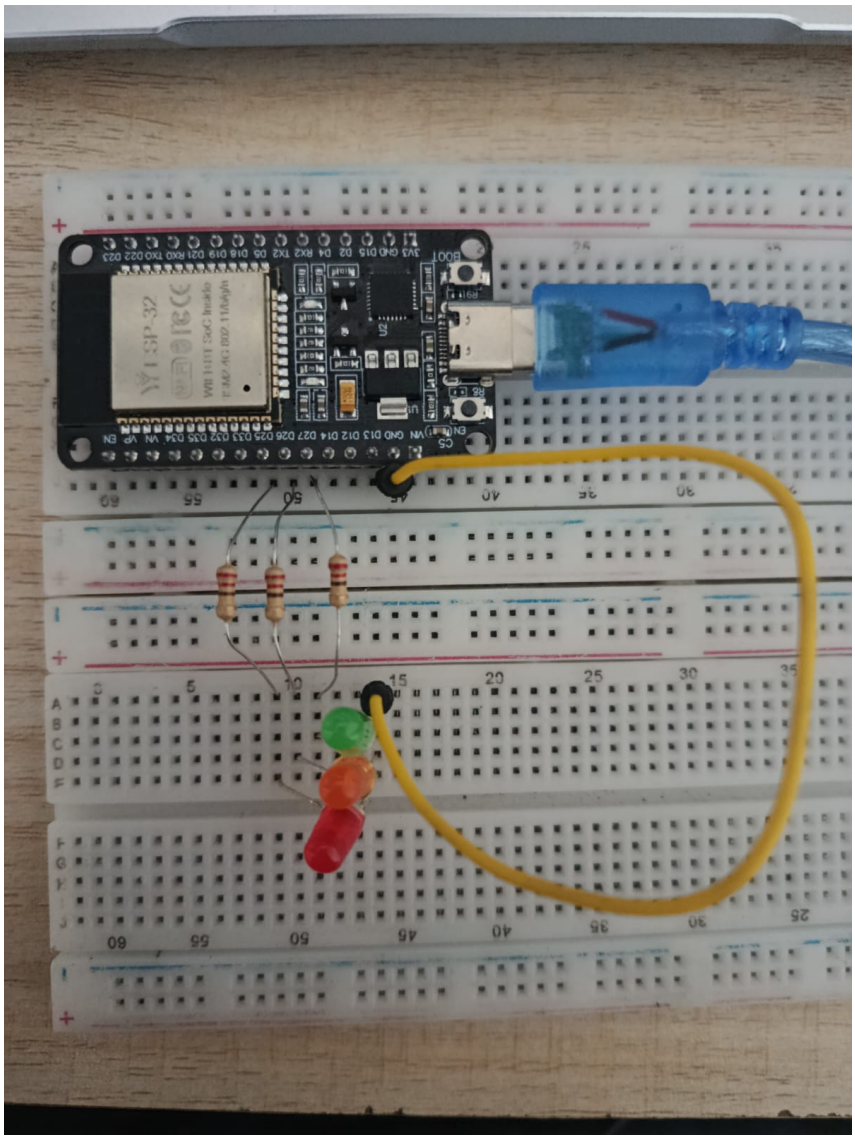

Figure 13: Breadboard wiring of the three LEDs.

6. **Write the glue layer.**
   Open `main.c` with `notepad main\main.c`. Before writing it, open your generated `components\scade_gen\operator1_module0.h` and read the exact signature of the step function and the name of the context type — the generator derives both from your module and operator names, so they will differ from the placeholders used here. In this build the generator produced the context type `outC_operator1_module0` and a step function that takes *all four* model outputs as pointer arguments, in the order `cont, green, yellow, red`, followed by the context. Note that `cont` (the tick counter) is exported as an output even though no LED uses it: it must still be passed to the step function, and it is convenient for the phase log. The glue configures GPIOs 25/26/27 as outputs (Practice 2 API), declares one variable per model output, calls the generated `reset` once, then loops: call the step function, copy the boolean outputs to the GPIOs with `gpio_set_level`, log the state, and delay 100 ms.

```
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "driver/gpio.h"
#include "esp_log.h"
```

```c
#include "operator1_module0.h"   // generated (TrafficLight)

static const char *TAG = "SCADE";
static outC_operator1_module0 ctx;
static swan_int32 cont;                    // tick counter output
static swan_bool green, yellow, red;

#define LED_RED    25
#define LED_YELLOW 26
#define LED_GREEN  27

void app_main(void)
{
    gpio_set_direction(LED_RED,    GPIO_MODE_OUTPUT);
    gpio_set_direction(LED_YELLOW, GPIO_MODE_OUTPUT);
    gpio_set_direction(LED_GREEN,  GPIO_MODE_OUTPUT);

    operator1_reset_module0(&ctx);
    while (1) {
        operator1_module0(&cont, &green, &yellow, &red, &ctx);
        gpio_set_level(LED_RED,    red);
        gpio_set_level(LED_YELLOW, yellow);
        gpio_set_level(LED_GREEN,  green);
        ESP_LOGI(TAG, "tick=%d  G=%d Y=%d R=%d",
                 (int)cont, (int)green, (int)yellow, (int)red);
        vTaskDelay(pdMS_TO_TICKS(100));
    }
}
```

Finally, declare the dependencies in `main\CMakeLists.txt`. Because the glue includes `driver/gpio.h`, the `esp_driver_gpio` component must be listed alongside `scade_gen`; omitting it fails the build with "`esp_driver_gpio ... is not in the requirements list of main`".

```
idf_component_register(SRCS "main.c"
                       INCLUDE_DIRS "."
                       REQUIRES scade_gen esp_driver_gpio)
```

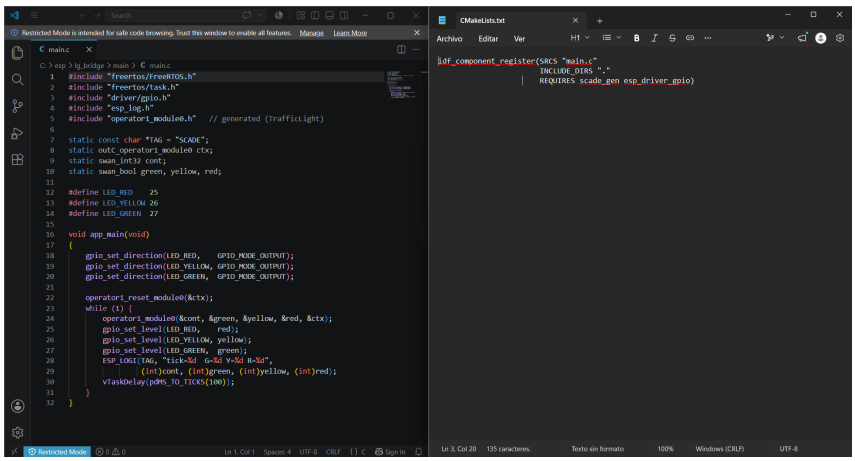

Figure 14: Glue layer mapping the model outputs to the GPIOs.

7. **Build, flash, and verify against the simulation.**
`idf.py -p COM8 flash monitor`. Read the phase transitions in the log: `G=1` for ticks 0–49, `Y=1` for 50–69, `R=1` for 70–119, then wrap. Using the monitor timestamps, confirm green lasts ≈5 s, yellow ≈2 s, red ≈5 s, and that the lit LED matches the `G/Y/R` columns and the step-3 simulation.

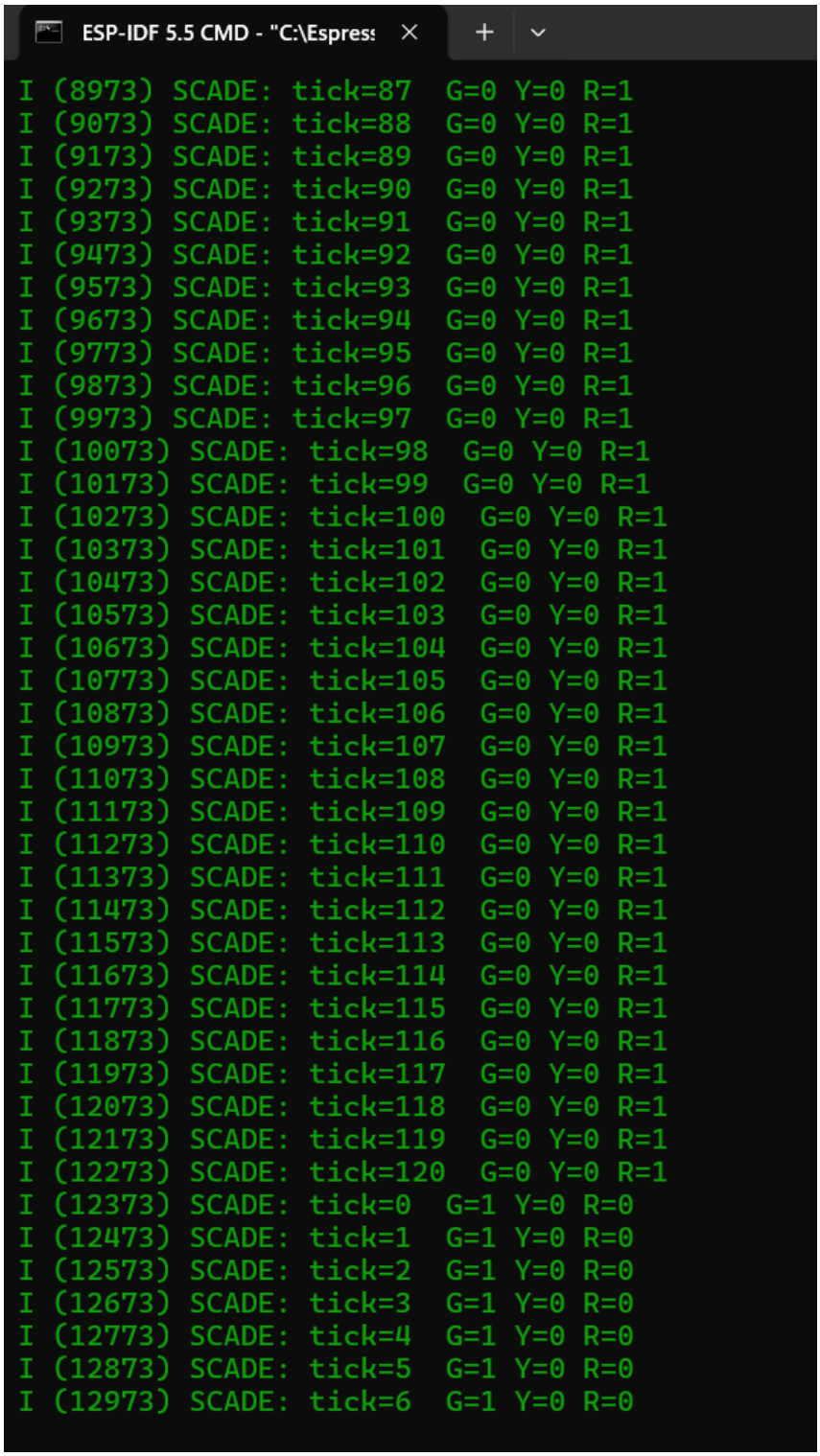

Figure 15: Monitor output showing the 5/2/5 s phase timing.

7 Expected Results

Expected Results

- **SCADE Simulation:** The simulator shows `cont` ramping 0–119 and wrapping, with `green` true for ticks 0–49, `yellow` for 50–69, `red` for 70–119, exactly one light true per cycle, repeating indefinitely.
- **Console / UART Output:** One line per model cycle (every 100 ms), showing the tick and the three boolean outputs. The active light changes at ticks 50, 70, and the wrap back to 0:

```
I (274) main_task:  Calling app_main()
I (284) SCADE: tick=0   G=1 Y=0 R=0
I (384) SCADE: tick=1   G=1 Y=0 R=0
...
I (5284) SCADE: tick=50  G=0 Y=1 R=0
...
I (7284) SCADE: tick=70  G=0 Y=0 R=1
...
I (12284) SCADE: tick=0   G=1 Y=0 R=0
```

- **Physical Behavior:** The three breadboard LEDs cycle green (5 s) → yellow (2 s) → red (5 s), with never two LEDs lit at once (verify against the `G/Y/R` columns in the log); pressing `EN` restarts the sequence from green at `tick=0`, demonstrating the `reset` contract.

8 Assessment Questionnaire

Answer each question based on your specific implementation. Justify your responses with references to your design decisions.

Q1. [*Comprehension*] Open your generated `operator1_module0.h`. Identify the context type name and the field that stores the `Initialized Unit Delay` memory of the tick counter (`mem1`). Then explain why the step function receives `cont, green, yellow, red` as pointer arguments rather than returning them, and why `cont` is present in the signature even though no LED is driven by it.

Q2. [*Analysis*] Your glue layer calls the step function every 100 ms, so the 12 s period is a property of the *caller*, not the model. Predict what would happen to the traffic-light timing — on the LEDs and in the monitor — if the delay in the glue loop were changed to 50 ms without touching the model, and explain why the SCADE One simulation would show no difference at all.

Q3. [*Analysis*] The glue layer you wrote depends on two ESP-IDF components declared in `main/CMakeLists.txt`: `scade_gen` and `esp_driver_gpio`. Explain what each dependency provides and what specific build error appears if each one is omitted. Then argue whether the type mapping in `swan_config.h` belongs conceptually to `scade_gen` or to `main`, and why.

Q4. [*Synthesis*] A new requirement adds a flashing-green warning phase: during the last second of green (ticks 40–49), `green` must alternate true/false every 5 ticks. Extend your `TrafficLight` model to satisfy this using only Practice 1 blocks, describe the change, and state what — if anything — must change in the generated header, the glue layer, and `swan_config.h` after regeneration.

Q5. [*Critical*] Suppose a teammate "fixes" a timing bug by editing the generated `operator1_module0.c` directly instead of changing the model and regenerating. Identify two concrete ways this breaks the model-based workflow established in this practice, and describe what evidence in the project — including the header banner emitted by the Swan generator — would reveal the edit during a review.

9 Conclusion

Write a concise conclusion reflecting on what you implemented, what you observed, and what challenges you encountered. Connect your findings to the broader context of model-based design for critical systems — in particular, what it means that the model, not the C code, is now the single source of truth for the system's behavior.

10 Bonus Challenge

★ Bonus Challenge

Challenge:
Add a night-mode input to the system without adding any block type you do not already know: give `TrafficLight` a `bool` input `night`; when true, the normal sequence is suspended and `yellow` flashes at 1 Hz (5 ticks on, 5 ticks off) while `red` and `green` stay off. In the glue layer, feed the input from the `BOOT` button (GPIO 0, active low) so holding the button switches the intersection to night mode live. Re-simulate, regenerate, re-integrate — the full loop, end to end.

Hint:
The flashing yellow is the counter you already have, viewed through a different comparison — think about what `(tick mod 10) < 5` *looks like over time, and let* `if-then-else` *select between the normal outputs and the night-mode outputs. Note that when the model gains an input, the generated step function gains an argument.*

11 Troubleshooting

This section is populated with real issues encountered during development. If you run into a problem not listed here, document it using the same format below.

Issue: Flash aborts: "Build directory configured for project X not Y"

Symptom: After creating a project by copying an existing one (e.g. `scade_counter` from `lg_env_check`), `idf.py flash` refuses to run: *Build directory configured for project 'C:\esp\lg_env_check' not 'C:\esp\scade_counter'.*

Cause: `xcopy /e` also copied the `build/` folder, whose CMake cache stores absolute paths to the original project.

Fix: Run `idf.py fullclean` once inside the new project (it deletes the stale `build/`), then build or flash normally. The first build after a fullclean recompiles everything and takes several minutes.

Issue: CMake error: "Cannot find source file: .../scade_gen/2"

Symptom: Configuration fails claiming a source file named `2` (or another bare number) does not exist, followed by "No SOURCES given to target".

Cause: The `CMakeLists.txt` content was copy-pasted from a PDF whose code listing included line numbers; the stray digits became CMake tokens and were interpreted as file names.

Fix: Open the file (`notepad components\scade_gen\CMakeLists.txt`), delete everything, and retype or paste the content *without* leading numbers. Verify with `type` that no digit starts a line.

Issue: Compile fails: "fatal error: swan_config.h: No such file or directory"

Symptom: Every generated source aborts at `swan_types.h:7` because `swan_config.h` is missing.

Cause: `swan_config.h` is *not* produced by the Swan code generator — `swan_types.h` is only a wrapper that includes it. The platform adaptation header must be written by the integrator.

Fix: Create `components\scade_gen\swan_config.h` with the complete ESP32 adaptation given in Worked Example step 5.

Issue: Compile fails: "unknown type name 'swan_int32"'

Symptom: `swan_config.h` exists but the generated code still fails on the `swan_*` types or the `swan_lit_*` literal macros.

Cause: The adaptation header is incomplete — it must define *both* the base type mapping (`swan_int32` → `int32_t`, ...) and the literal constructor macros (`swan_lit_int32(v)`, ...) used by every constant in the generated code.

Fix: Use the complete `swan_config.h` from Worked Example step 5; if the compiler reports another missing `swan_*` identifier, add its typedef or macro to the same file following the same pattern.

Issue: Link fails with "undefined reference" to generated symbols

Symptom: The build compiles `main.c` but the link stage fails with undefined references to constants or type-support functions of the generated code.

Cause: The component's `CMakeLists.txt` lists only `operator0_module0.c`, but `code/` also ships `swan_consts.c` and `swan_types.c`; any `.c` not listed in `idf_component_register` is silently ignored (Practice 2, Section 4).

Fix: List every generated `.c` file in `SRCS`. Headers need no entry.

Issue: Configure fails: "The component 'scade_gen' could not be found"

Symptom: `idf.py build` aborts during configuration stating that the `scade_gen` component cannot be found, even though the generated files were copied.

Cause: Two variants. Either the `components\scade_gen` folder was created outside the project (a missing `cd lg_bridge` left it under `C:\esp\components`), or the folder exists but has no `CMakeLists.txt` — ESP-IDF only treats a directory as a component when it contains one.

Fix: Confirm the location with `dir C:\esp\lg_bridge\components\scade_gen`. Move it into the project if it is misplaced, and create the component's `CMakeLists.txt` with an `idf_component_register` listing every generated `.c`.

Issue: Compile fails: "esp_driver_gpio is not in the requirements list of main"

Symptom: Compilation of `main.c` stops with a message that `driver/gpio.h` is provided by `esp_driver_gpio`, which is not in the requirements of `main`.

Cause: The glue layer includes `driver/gpio.h` to drive the LEDs, but `main/CMakeLists.txt` only declared `scade_gen`. Unlike the counter (which used no driver), the traffic light needs GPIO.

Fix: Add `esp_driver_gpio` to the `REQUIRES` list: `REQUIRES scade_gen esp_driver_gpio`.

Issue: One LED never lights, or two LEDs light at once

Symptom: The console log shows the expected `G/Y/R` pattern (exactly one output `1` per cycle), but on the breadboard a color stays off or two colors are lit together.

Cause: Wiring, not logic. A reversed LED, a floating GPIO signal wire, or a missing common-ground connection. The log is the reference: if `G=1` but green is dark, the fault is physical.

Fix: Verify each color has its own signal wire to GPIO 25/26/27 through a 220 Ω resistor, all cathodes tied to a common GND rail, and that GND is wired back to the ESP32. Cross-check the lit LED against the `G/Y/R` columns.

Issue: Cannot type commands: the terminal is captured by the monitor

Symptom: After `idf.py flash monitor`, the terminal will not accept new commands — keystrokes go to the running monitor.

Cause: `idf_monitor` owns the console until it is exited; it is not a hang.

Fix: Press `Ctrl+]` to quit the monitor and return to the prompt (on a Spanish keyboard the `]` is `AltGr+`` ; `Ctrl+T` then `Ctrl+X` also works). The ESP32 keeps running the last flashed firmware.